

1. Single Inheritance – Scenario

1. How would you implement this using single inheritance?

- Create a Vehicle class.
- Create a Car class that extends Vehicle.
- Car inherits all methods from Vehicle.

2. Should Car override startEngine()? Why?

- To log the engine start time.
- It can add new functionality while keeping the original behavior.

3. How can the parent class implementation still be executed?

- Use the **super** keyword.
- `super.startEngine();`

4. What happens if Car does not override startEngine()?

- The **Vehicle** class method executes.
- No engine start time will be logged.

2. Multilevel Inheritance – Scenario

1. Why is multilevel inheritance suitable here?

- It allows SeniorDeveloper to inherit features from both Employee and Developer.
- No code duplication.

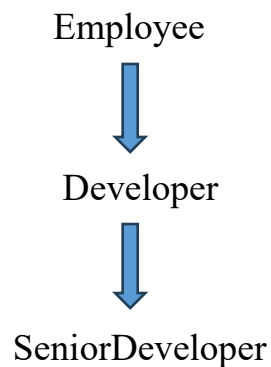
2. Which members are accessible in SeniorDeveloper?

- Employee ID
- Employee Name
- Programming Language
- Team Size
- Code Review methods

1. If Developer overrides a method from Employee, which implementation will SeniorDeveloper inherit?

- It inherits the Developer's overridden method.

2. Draw the inheritance hierarchy.



3. Hierarchical Inheritance – Scenario

1. Why is hierarchical inheritance appropriate here?

- Multiple classes share the same parent.
- Common features are inherited.

1. Where should common validation logic be implemented?

- In the Payment class.

2. How can each payment class provide its own transaction processing?

- By overriding the processPayment() method.

3. What is the advantage over writing three independent classes?

- Code reuse
- Less duplication
- Easier maintenance

4. Compile-Time Polymorphism (Method Overloading) – Scenario

1. Which OOP concept should be used?

- Method Overloading

2. How will Java decide which method to invoke?

- Based on the number, type, and order of parameters.

3. Can methods be overloaded by changing only the return type?

- No.

4. What happens if add(5, 10.0) is called?

- Java calls the method that matches (int, double) or (double, double) after type conversion.

5. Runtime Polymorphism (Method Overriding) – Scenario

1. Which OOP concept is being used?

- Method Overriding (Runtime Polymorphism)

2. Which version of calculateDeliveryTime() executes?

- The method of the actual object (Pizza, Burger, or Biryani).

3. When is the method selection decided?

- At runtime.

4. What would happen if calculateDeliveryTime() were declared static?

- Static methods cannot be overridden.
- The parent class method is chosen based on the reference type, not the object.

6. Interface – Scenario

1. Would you use an interface or inheritance? Why?

- Static methods cannot be overridden.
- The parent class method is chosen based on the reference type, not the object.

2. Design an appropriate interface.

- interface SmartDevice {

```
void turnOn();  
  
void turnOff();  
  
}
```

3. Can one device implement multiple interfaces? Give an example.

- class SmartAC implements SmartDevice, WifiEnabled

```
{  
  
}
```

4. Why is an interface better than creating a common base class in this scenario?

- Supports multiple inheritance of behavior.
- More flexible.
- Easy for third-party manufacturers to implement.
- Reduces dependency between classes.